

Distributed NMPC for Cooperative Aerial Manipulation of Cable-Suspended Loads

Nicola De Carli , Riccardo Belletti, Emanuele Buzzurro, Andrea Testa , Giuseppe Notarstefano , *Member, IEEE*, and Marco Tognon , *Member, IEEE*

Abstract—In this letter, we address the problem of cooperative manipulation of a cable-suspended load by a team of aerial robots. Unlike classical approaches that rely on centralized controllers, we propose a *Distributed Nonlinear Model Predictive Control* (DNMPC) framework in which the UAVs communicate over a peer-to-peer network a reduced amount of variables. In the proposed method, each robot handles only a small subset of the global optimization problem. The optimal motion computed by the DNMPC loop is then used as a reference for local nonlinear controllers that track the trajectory and compute the robot's actuation inputs. We validate the proposed scheme both through numerical simulations and real-world experiments on the *Fly-Crane* system: a rigid platform connected to three robots by pairs of cables.

Index Terms—Aerial systems, multi-robot systems, cooperating robots.

I. INTRODUCTION

IN recent years, there has been increasing interest in using multiple uncrewed aerial vehicles (UAVs) for manipulation and transportation tasks [1]. Multi-UAV systems offer advantages over single-robot solutions, such as greater payload capacity and full six-degree-of-freedom (6DoF) control of large objects [2]. Applications include search-and-rescue [3], construction [4], and industrial transportation [5].

To enable UAV teams to manipulate and transport payloads, researchers have explored different attachment mechanisms, including spherical joints [6] and cables [5], [7], [8]. Cable-based systems are particularly attractive thanks to their lightweight nature and cost-effectiveness. Achieving full pose control requires generating a six-dimensional wrench through coordinated adjustments of cable forces and orientations. A minimal configuration consists of three UAVs connected to non-collinear points on the payload [5]. However, to compensate for external wrenches,



Fig. 1. Fly-Crane system: A cooperative aerial transport system composed of three quadrotors connected to a payload via cables, enabling controlled manipulation and transportation.

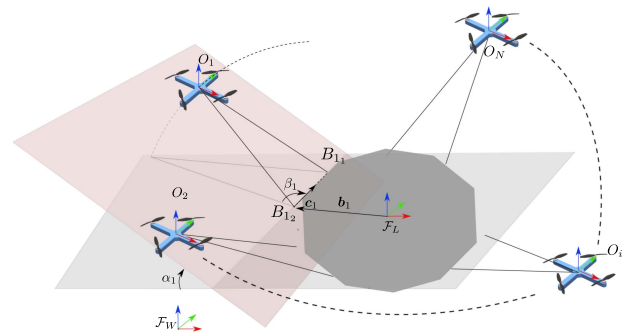


Fig. 2. Schematic representation of the Fly-Crane system with N robots and its key variables. The pink plane illustrates the plane formed by the two cables attached to robot 1, while the gray plane corresponds to the xy -plane of the load.

such as wind disturbances, this setup requires repositioning the UAVs, leading to slow and imprecise responses.

A force-closed design [9] mitigates this issue by enabling rapid compensation of external forces through direct tension adjustments while maintaining cable orientation. This property requires at least six cables. The Fly-Crane system [10] (shown in Figs. 1 and 2) achieves this by equipping each UAV with two cables, ensuring force closure with a minimal number of robots. While the standard Fly-Crane setup employs three UAVs, the concept can scale to larger teams, increasing load capacity, disturbance rejection, and redundancy while preserving the benefits of force closure [10].

Classical approaches to cooperative transportation often rely on centralized control, where a single computation unit generates planning and control inputs for all robots. These methods require solving large-scale optimization problems and demand high computational and communication resources. As a result,

Received 31 March 2025; accepted 4 August 2025. Date of publication 1 September 2025; date of current version 5 September 2025. This article was recommended for publication by Associate Editor D. Aksaray and Editor M. Ani Hsieh upon evaluation of the reviewers' comments. This work was supported by the Project ANR-23-CE33-0013 "FlyHandyBot". (Nicola De Carli and Riccardo Belletti contributed equally to this work.) (Corresponding author: Nicola De Carli.)

Nicola De Carli, Riccardo Belletti, Emanuele Buzzurro, and Marco Tognon are with Univ Rennes, CNRS, Inria, IRISA, F-35000 Rennes, France (e-mail: ndc@kth.se; marco.tognon@inria.fr).

Andrea Testa and Giuseppe Notarstefano are with the Department of Electrical, Electronic and Information Engineering, Alma Mater Studiorum - Università di Bologna, 40126 Bologna, Italy (e-mail: a.testa@unibo.it; giuseppe.notarstefano@unibo.it).

This article has supplementary downloadable material available at <https://doi.org/10.1109/LRA.2025.3604703>, provided by the authors.

Digital Object Identifier 10.1109/LRA.2025.3604703

2377-3766 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See <https://www.ieee.org/publications/rights/index.html> for more information.

they are typically executed offboard on high-performance computers and rely on high-bandwidth networks. To address these limitations, distributed optimization has emerged as an effective paradigm for multi-robot systems [11], [12]. In this letter, we propose a Distributed Nonlinear Model Predictive Control (DNMPC) framework for collaborative aerial transportation, where a team of UAVs, connected to a load via cables, coordinates in a fully distributed manner.

A. Related Work

Various control strategies have been proposed for cooperative aerial transportation. Centralized approaches include reactive methods [13], [14], which do not explicitly handle constraints, and predictive strategies such as NMPC [7], [15], which optimize trajectories for the full system but require significant computational and communication resources.

To improve scalability, decentralized and distributed strategies have been explored. Some decentralized methods rely on formation control [16], [17] and leader-follower schemes [18], [19] to stabilize the load to a certain configuration rather than tracking a trajectory.

For the Fly-Crane system, previous works have primarily used reactive strategies, including robust H_∞ control [20] to deal with parameter uncertainties and an admittance framework combined with an inverse kinematic controller [21] to ensure safe task execution in presence of expected or unexpected interactions between the payload and the environment.

Distributed MPC has been investigated for aerial transportation, such as in [22], where quadrotors use rigid rods, though requiring all-to-all communication. In contrast, [23] presents a linear decentralized MPC for quasi-static motions of a bar, hence, not requiring control of the full 6-dimensional pose.

Beyond aerial robotics, distributed optimization has been explored in related domains. For instance, [24] investigates distributed transportation with maritime vessels using a multi-layer control structure. While some stages of the approach are distributed, a central coordinator is required to properly update the trajectory. In [25], the authors apply distributed optimization to trajectory planning for contact-based manipulation, requiring consensus on the full force vector applied by each robot. This consensus mechanism increases computational complexity as the number of robots grows. These studies demonstrate the potential of distributed optimization and MPC techniques in cooperative transportation while also highlighting limitations related to scalability and single points of failure of current approaches.

B. Contributions

This work proposes a Distributed Nonlinear Model Predictive Control (DNMPC) framework for tracking the full pose of a cable-suspended load using a team of UAVs. Each UAV receives reference trajectories—position, velocity, and acceleration—for its low-level controller. The optimal control problem is solved via a partition-based ADMM scheme [26], [27], [28], in which each UAV optimizes a local subproblem in a peer-to-peer network without a central coordinator. While such methods have

been studied in theory and in generic contexts, we tailor the algorithm to the cooperative transportation problem by exploiting its specific sparsity structure: UAVs are coupled only through the shared load. This leads to an efficient and scalable implementation, where the optimization size per UAV remains independent of the team size—unlike in centralized MPC.

The partition-based distributed ADMM NMPC algorithm we use was introduced in [28], and a preliminary version of it had been used for contact-implicit cooperative trajectory optimization in [25]. However, [25] required each robot to optimize the entire vector of forces applied by all robots to the load, thus limiting its scalability. In contrast, our approach considers only the kinematic model of the load, requiring robots to exchange only the trajectory of the load with their neighbors. This significantly reduces communication and ensures the algorithm complexity remains independent of the number of robots.

This work brings a largely theoretical distributed optimal control algorithm into practice, overcoming challenges posed by model uncertainties and hardware constraints. We validate the approach on the Fly-Crane platform through both realistic Software-in-the-Loop simulations and real-world experiments.

II. PRELIMINARIES

A. Problem Formulation

We consider a system composed of N UAVs attached to a rigid body (load) by cables. In particular, we refer to the Fly-Crane model (Fig. 2), which, as previously mentioned, exhibits the favourable robustness property of force closure, allowing for precise kinematic control. However, we underline that the same methodology could also be applied to classical configurations with one cable per robot.

In order to describe the system, let us define an inertial frame $\mathcal{F}_W = \{O_W, x_W, y_W, z_W\}$, a body frame $\mathcal{F}_i = \{O_i, x_i, y_i, z_i\}$ attached to the center of mass of the i -th robot and a body frame $\mathcal{F}_L = \{O_L, x_L, y_L, z_L\}$ attached to the center of mass of the load, where O_* and $\{x_*, y_*, z_*\}$ are the origin and unit axis of the frame $\mathcal{F}_*$. The vector $Wp_L \in \mathbb{R}^3$ describes the position of O_L with respect to $\mathcal{F}_W$ and the unit quaternion $Wq_L = [q_w \ q_x \ q_y \ q_z]^\top \in \mathbb{S}^3$ describes the orientation of $\mathcal{F}_L$ with respect to $\mathcal{F}_W$.¹ Quaternion multiplication will be represented by matrix multiplication notations, namely $q_1 \odot q_2 = Q(q_1)q_2$, where $\odot$ indicates the quaternion multiplication, and $Q(\cdot) : \mathbb{S}^3 \rightarrow \mathbb{R}^{4 \times 4}$. The corresponding rotation matrix, denoted as $R_L \in SO(3)$, can be obtained as a function of q_L . Moreover, we use the superscript d to indicate the desired value of a variable to be tracked, e.g., p_L^d denotes the desired load position.

Each UAV is tethered to the load by two cables that link the UAV's center of mass to an arbitrary point on the load. These connections are designed to ensure that there are no rotational constraints between the cable-platform and cable-UAV interfaces. As it is common, we assume that the cables have negligible mass and inertia with respect to the other bodies of the system,

¹The left superscript indicates the reference frame. From now on, $\mathcal{F}_W$ is considered as reference frame when the superscript is omitted.

the deformations due to elasticity are negligible in the operative conditions and the cables are always taut. As a consequence, the k -th cable is characterized by a constant length $l_k \in \mathbb{R}_{\geq 0}$. These assumptions are introduced to simplify the model and enable a tractable control design. Despite the mismatch with real cable behavior, the effectiveness of the resulting controller is validated through experiments.

We assume that the UAVs are able to communicate among them. In particular, we assume that the communication interaction among the robots is described by a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \{1, 2, \dots, N\}$ and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ are the vertex set and the edge set, respectively. More in detail, $(i, j) \in \mathcal{E}$ if UAV i can communicate with UAV j . We denote as $\mathcal{N}_i$ the set of robots which communicate with robot i . Furthermore, we assume the communication graph $\mathcal{G}$ to be connected.

B. Modeling

Robot model: The position and velocity of the i -th robot are denoted as $\mathbf{p}_i \in \mathbb{R}^3$ and $\mathbf{v}_i \in \mathbb{R}^3$, respectively. A low-level controller is assumed to be in place, capable of tracking position trajectories while accounting for the robot's full dynamics, e.g., [29], [30]. Thus, we model the closed-loop translational dynamics of the robot as a double integrator:

$$\begin{aligned} \dot{\mathbf{p}}_i &= \mathbf{v}_i \\ \dot{\mathbf{v}}_i &= \mathbf{a}_i, \end{aligned} \quad (1)$$

in which the acceleration $\mathbf{a}_i \in \mathbb{R}^3$ is the input to the robot. This model provides a reasonable approximation for both fully-actuated vehicles and underactuated ones, assuming motions with small jerk and snap.

Load model: The load dynamics are modeled as a kinematic system in the form:

$$\begin{aligned} \dot{\mathbf{p}}_L &= \mathbf{v}_L, \\ \dot{\mathbf{q}}_L &= \frac{1}{2} \mathbf{Q}(\mathbf{q}_L) \begin{bmatrix} 0 \\ L\boldsymbol{\omega}_L \end{bmatrix}, \end{aligned} \quad (2)$$

where $\mathbf{v}_L$ and $L\boldsymbol{\omega}_L$ are, respectively, the linear and angular velocity of the load, with the latter being expressed in the load body frame.

Cable configuration: In the Fly-Crane configuration, each robot is subject to two geometric constraints, restricting its movement relative to the load to a circular trajectory parameterized by the angle $\alpha_i \in (-\pi, \pi]$ (see Fig. 2), whose kinematics are represented by a single integrator:

$$\dot{\alpha}_i = \omega_{\alpha_i}. \quad (3)$$

The resulting geometric constraint can be written as:

$$\mathbf{p}_i = \mathbf{p}_L + \mathbf{R}_L L\mathbf{p}_i(\alpha_i), \quad (4)$$

where $L\mathbf{p}_i(\alpha_i) = l_i \sin(\beta_i) \mathbf{R}_{Lc_i}(\alpha_i) \frac{L\mathbf{b}_i}{\|L\mathbf{b}_i\|}$. Here, β_i is the fixed angle between the segment connecting the drone-cable attachment point O_i to the load-cable attachment point B_{i2} , and the segment connecting the two load-cable attachment points B_{i1} and B_{i2} (see Fig. 2). Additionally, $\mathbf{c}_i$ represents the unit vector between the two load-cable attachment points, while $\mathbf{b}_i$ denotes the relative displacement between the load center of mass and

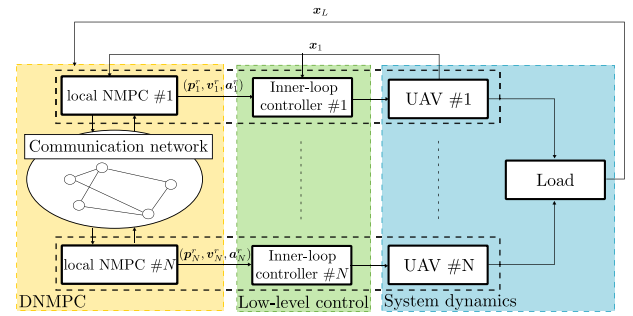


Fig. 3. Diagram of the proposed control framework.

B_{i2} . For the system to be more robust to disturbances as well as to avoid inter-robot collisions, we want the cable angles to operate in a certain region

$$\underline{\alpha}_i \leq \alpha_i \leq \overline{\alpha}_i, \quad (5)$$

with $\underline{\alpha}_i > 0$. We express the kinematic relationship linking the robot velocities to the payload velocity twist and cable angular rates:

$$\mathbf{v} = \mathbf{J}(\mathbf{q}_L, \boldsymbol{\alpha}) \begin{bmatrix} \mathbf{v}_L \\ L\boldsymbol{\omega}_L \end{bmatrix}, \quad (6)$$

where $\mathbf{v} = [\{\mathbf{v}_i\}_{i=1}^N]$ and $\boldsymbol{\omega}_\alpha = [\{\omega_{\alpha_i}\}_{i=1}^N]$. We use the notation $[\{\mathbf{v}_i\}_{i=1}^N] = [\mathbf{v}_1^\top \ \mathbf{v}_2^\top \ \dots \ \mathbf{v}_N^\top]^\top$ to denote the vertical stack of vectors indexed by the integer $i \in \{1, 2, \dots, N\}$. Moreover, we use $\mathbf{J}(\mathbf{q}_L, \boldsymbol{\alpha})$ to denote the geometric jacobian matrix, which, for $N \geq 3$, is full column rank (cf. [31]). As a result, given robot velocities that satisfy the system's geometric constraints, the load velocity twist and cable angular velocities are uniquely determined.

Objective: The control objective is to ensure that the load's full pose—both position and orientation—tracks the desired trajectory $(\mathbf{p}_L^d, \mathbf{q}_L^d, \mathbf{v}_L^d, \boldsymbol{\omega}_L^d)$ in a distributed manner.

III. CONTROL METHODOLOGY

In this section, we describe the proposed control strategy, which includes a distributed NMPC as the outer-loop controller and a low-level controller which tracks the reference trajectory provided by the NMPC (see Fig. 3). We first describe the global optimal control problem in its centralized form and then explain how it can be reformulated in a distributed manner using *partition-based ADMM*.

A. OCP Formulation

In the optimal control problem formulation, the control inputs include the robots' linear accelerations and the angular velocities of the cables, grouped as $\mathbf{u}_i = [\mathbf{a}_i^\top \ \omega_{\alpha_i}^\top]^\top$, as well as the load's linear and angular velocities, denoted by $\mathbf{u}_L = [\mathbf{v}_L^\top \ L\boldsymbol{\omega}_L^\top]^\top$. We define the vector containing all system inputs as

$$\mathbf{u} = \begin{bmatrix} \{\mathbf{u}_i\}_{i=1}^N \\ \mathbf{u}_L \end{bmatrix}. \quad (7)$$

The state is given by the robots position, velocity and corresponding cable state $\mathbf{x}_i = [\mathbf{p}_i^\top \ \mathbf{v}_i^\top \ \alpha_i]^\top$ as well as the load pose $\mathbf{x}_L = [\mathbf{p}_L^\top \ \mathbf{q}_L^\top]^\top$.

We point out that the platform velocity twist $\mathbf{u}_L$ and cable velocities ω_{α_i} are fictitious inputs. They are not actually a controlled input, but an algebraic function of the robots velocities, due to the geometric constraint (4). Consistency among the resulting velocities is ensured by enforcing (4) inside the following nonlinear optimal control problem (OCP):

$$\begin{aligned} \min_{\mathbf{u}(\cdot)} \quad & \int_t^{t+T} \left[N \ell_L(\mathbf{x}_L(\tau), \mathbf{u}_L(\tau)) \right. \\ & \left. + \sum_{i=1}^N \ell_i(\mathbf{x}_i(\tau), \mathbf{u}_i(\tau)) \right] d\tau \\ & + N \phi_L(\mathbf{x}_L(t+T)) + \sum_{i=1}^N \phi_i(\mathbf{x}_i(t+T)) \\ \text{s.t.} \quad & \text{Initial conditions:} \\ & \mathbf{x}_L(t) = \hat{\mathbf{x}}_L(t) \quad \mathbf{x}_i(t) = \hat{\mathbf{x}}_i(t) \\ & \text{System dynamics:} \quad (1) - (3) \\ & \text{Load dynamics:} \quad (2) \\ & \text{Geometric constraint:} \quad (4) \\ & \text{Inequality constraints:} \quad (5) \\ & \mathbf{p}_i(\tau) \notin \mathcal{O}(\tau) \quad \mathbf{v}_i(\tau) \in \mathcal{V}_i, \quad \mathbf{a}_i(\tau) \in \mathcal{A}_i \\ & \forall \tau \in [t, t+T] \quad i = 1, \dots, N \end{aligned} \quad (8)$$

where T is the prediction horizon, the factor N in the cost is added in view of later developments, $\hat{\mathbf{x}}_L$ and $\hat{\mathbf{x}}_i$ are the load and robots state feedback at time t , $\mathcal{O}(t)$ represents the space occupied by obstacles at time t , $\mathcal{V}_i$ and $\mathcal{A}_i$ represent the convex sets in which velocity and accelerations must lie due to saturations, respectively. The terms $\ell_L(\cdot)$ and $\ell_i(\cdot)$ are stage costs, accounting for load tracking error and local variables specific to each robot, respectively.

$$\begin{aligned} \ell_L(\mathbf{x}_L, \mathbf{u}_L) &= \|\mathbf{p}_L^d - \mathbf{p}_L\|_{\mathbf{P}_p}^2 + \|\mathbf{e}_{q_L}\|_{\mathbf{P}_q}^2 + \|\mathbf{v}_L^d - \mathbf{v}_L\|_{\mathbf{P}_v}^2 \\ &+ \|\mathbf{L}\omega_L^d - \mathbf{L}\omega_L\|_{\mathbf{P}_\omega}^2 \\ \ell_i(\mathbf{x}_i, \mathbf{u}_i) &= \|\alpha_i^d - \alpha_i\|_{\mathbf{P}_{\alpha_i}}^2 + \|\omega_{\alpha_i}^d - \omega_{\alpha_i}\|_{\mathbf{P}_{\omega_{\alpha_i}}}^2 \\ &+ \|\mathbf{u}_i^d - \mathbf{u}_i\|_{\mathbf{P}_{u_i}}^2, \end{aligned} \quad (9)$$

with $\mathbf{P}_p, \mathbf{P}_q, \mathbf{P}_v, \mathbf{P}_\omega, \mathbf{P}_{\alpha_i}, \mathbf{P}_{\omega_{\alpha_i}}$ and $\mathbf{P}_{u_i}$ being positive definite weight matrices of appropriate dimensions, and $\mathbf{e}_{q_L}$ being the quaternion error, while $\phi_L(\cdot)$ and $\phi_i(\cdot)$ are the respective terminal costs:

$$\begin{aligned} \phi_L(\mathbf{x}_L, \mathbf{u}_L) &= \|\mathbf{p}_L^d - \mathbf{p}_L\|_{\mathbf{P}_{pT}}^2 + \|\mathbf{e}_{q_L}\|_{\mathbf{P}_{qT}}^2 \\ \phi_i(\mathbf{x}_i, \mathbf{u}_i) &= \|\alpha_i^d - \alpha_i\|_{\mathbf{P}_{\alpha_i T}}^2. \end{aligned} \quad (10)$$

The OCP (8) is transcribed into a nonlinear program (NLP) using a multiple-shooting approach, where the input trajectory is parameterized in a finite-dimensional space. We define a sequence of control times t_h separated by intervals of duration $\Delta t = T/H$, where H denotes the number of steps in the prediction horizon. The input is parameterized as piecewise constant: $\mathbf{u}(\tau) = \mathbf{u}_h(t)$, $\tau \in [t + t_h, t + t_{h+1})$. We then define the discretized load input trajectory which starts at time t as $\bar{\mathbf{u}}_L = [\mathbf{u}_{L,0}^\top, \dots, \mathbf{u}_{L,(H-1)}^\top]^\top$, and, similarly, the load state trajectory as $\bar{\mathbf{x}}_L$, the i -th robot input and state trajectories, respectively, as $\bar{\mathbf{u}}_i$ and $\bar{\mathbf{x}}_i$. Problem (8) contains terms that couple the trajectories of all the robots through the load and the geometric constraints, so that its centralized solution requires an update that uses all the agent trajectories. In the next section, we show how to distribute the problem so that each robot needs to compute only its own state and input trajectories, along with those of the load. Consistency among the load trajectories predicted by different robots is then enforced through a consensus-like mechanism.

B. Distributed OCP Formulation

Problem (8) is inherently centralized and not directly separable, as all robots must agree on the optimal load trajectory $(\bar{\mathbf{x}}_L, \bar{\mathbf{u}}_L)$. To enable a distributed formulation, we follow a partition-based implementation of the ADMM algorithm, as proposed in [26], [27], [28], adapting it to the specific structure of cooperative transportation. In particular, we exploit the sparsity induced by the fact that agents are only coupled through the shared load, allowing an efficient and scalable distributed formulation.

ADMM follows a decomposition-coordination approach, where a large global optimization problem is split into smaller subproblems that are solved locally, while a coordination mechanism ensures consistency across all solutions [32]. Specifically, the distributed ADMM reformulates the centralized problem by introducing auxiliary variables, enabling each robot to maintain a local copy of the shared optimization variables—in this case, the load trajectory. At each iteration, the algorithm consists of two main steps:

- *Local optimization* (decomposition step): Each robot i solves a small-scale local primal optimization problem.
- *Consensus update* (coordination step): A dual variable update is performed based on local solutions exchanged among neighboring robots. This step enforces agreement between neighboring copies of the shared variables, ensuring consistency across all robots.

In our formulation, each robot maintains a local copy of the load state $(\bar{\mathbf{x}}_L)$ and input $(\bar{\mathbf{u}}_L)$ trajectories, denoted as $\hat{\mathbf{x}}_{Li} = [\mathbf{x}_{Li,0} \ \dots \ \mathbf{x}_{Li,H-1}]$ and $\hat{\mathbf{u}}_{Li} = [\mathbf{u}_{Li,0} \ \dots \ \mathbf{u}_{Li,H-1}]$, respectively. A key feature of the ADMM-based approach is that these local copies are updated iteratively through communication with neighboring robots. If the algorithm converges, all robots reach agreement on their respective copies, ensuring consistency in the load trajectory. To enforce this agreement, we introduce a coupling constraint for each pair $(i, j) \in \mathcal{E}$, which enforces consistency among the local copies of the load

trajectory variables

$$\mathbf{u}_{Li,h} = \mathbf{u}_{Lj,h}, \quad \forall (i,j) \in \mathcal{E}, \text{ for } h \in \{0, 1, \dots, H-1\}. \quad (11)$$

To be able to split the problem in local subproblems, we introduce the auxiliary variables $\bar{\mathbf{c}}_{ij} = [\mathbf{c}_{ij,0} \ \dots \ \mathbf{c}_{ij,H-1}]$, with $\mathbf{c}_{ij,h} \in \mathbb{R}^6$ and we rewrite (11) as

$$\mathbf{u}_{Li,h} = \mathbf{c}_{ij,h} \quad \mathbf{c}_{ij,h} = \mathbf{c}_{ji,h} \quad \mathbf{u}_{Lj,h} = \mathbf{c}_{ji,h}. \quad (12)$$

The problem (8), after transcription to an NLP and introduction of auxiliary variables and associated constraint (12) becomes

$$\begin{aligned} \min_{\substack{\hat{\mathbf{u}}_{Li}, \hat{\mathbf{u}}_i, \\ \hat{\mathbf{x}}_{Li}, \hat{\mathbf{x}}_i \\ \mathbf{c}_{ij,h}, \mathbf{c}_{ji,h} \forall j \in \mathcal{N}_i \\ \forall i \in \{1, \dots, N\}}} & \quad \sum_{i=1}^N J_i(\hat{\mathbf{x}}_{Li}, \hat{\mathbf{u}}_{Li}, \hat{\mathbf{x}}_i, \hat{\mathbf{u}}_i) \\ \text{s.t.} & \quad \mathbf{x}_{i,(h+1)} = \mathbf{f}_i(\mathbf{x}_{i,h}, \mathbf{u}_{i,h}) \\ & \quad \mathbf{x}_{Li,(h+1)} = \mathbf{f}_L(\mathbf{x}_{Li,h}, \mathbf{u}_{Li,h}) \\ & \quad \mathbf{p}_{i,h} = \mathbf{p}_{Li,h} + \mathbf{R}(\mathbf{q}_{Li,h}) \mathbf{L} \mathbf{p}_i(\alpha_{i,h}) \\ & \quad \underline{\alpha}_i \leq \alpha_{i,h} \leq \bar{\alpha}_i \\ & \quad \mathbf{p}_{i,h} \notin \mathcal{O}(h), \mathbf{v}_{i,h} \in \mathcal{V}_i, \mathbf{a}_{i,h} \in \mathcal{A}_i \\ & \quad \mathbf{u}_{Li,h} = \mathbf{c}_{ij,h} = \mathbf{c}_{ji,h} \quad \forall j \in \mathcal{N}_i \\ & \quad \text{for } h = 0, \dots, H-1, \\ & \quad \forall i \in \{1, \dots, N\} \end{aligned} \quad (13)$$

where

$$\begin{aligned} J_i(\hat{\mathbf{x}}_{Li}, \hat{\mathbf{u}}_{Li}, \hat{\mathbf{x}}_i, \hat{\mathbf{u}}_i) \\ = \sum_{h=0}^{H-1} [\ell_{L,h}(\hat{\mathbf{x}}_{Li,h}, \hat{\mathbf{u}}_{Li,h}) + \ell_{i,h}(\hat{\mathbf{x}}_{i,h}, \hat{\mathbf{u}}_{i,h})], \end{aligned} \quad (14)$$

with $\ell_{L,h}$ and $\ell_{i,h}$ obtained by discretizing (9) and, similarly for $\mathbf{f}_i$ and $\mathbf{f}_L$, which are obtained by discretizing (1) and (2), respectively.

The problem in (13) has a structure amenable to be split in local subproblems and be solved using distributed ADMM. In the following, first, we introduce the augmented Lagrangian relaxation of problem (13) corresponding to the constraint (12). From this, the corresponding steps of the ADMM iterations are derived: in the first step, the primal problem [32] is solved; in the second step, after communication, the dual variables are updated. The augmented Lagrangian takes the following form:

$$\begin{aligned} \mathcal{L} = & \sum_{i=1}^N \left[J_i(\hat{\mathbf{x}}_{Li}, \hat{\mathbf{u}}_{Li}, \hat{\mathbf{x}}_i, \hat{\mathbf{u}}_i) \right. \\ & + \sum_{j \in \mathcal{N}_i} (\boldsymbol{\lambda}_{ij}^\top (\hat{\mathbf{u}}_{Li} - \bar{\mathbf{c}}_{ij}) + \boldsymbol{\mu}_{ij}^\top (\hat{\mathbf{u}}_{Li} - \bar{\mathbf{c}}_{ji})) \\ & \left. + \frac{\rho}{2} \|\hat{\mathbf{u}}_{Li} - \bar{\mathbf{c}}_{ij}\|^2 + \frac{\rho}{2} \|\hat{\mathbf{u}}_{Li} - \bar{\mathbf{c}}_{ji}\|^2 \right) \\ = & \sum_{i=1}^N \mathcal{L}_i(\hat{\mathbf{x}}_{Li}, \hat{\mathbf{u}}_{Li}, \hat{\mathbf{x}}_i, \hat{\mathbf{u}}_i, \{\bar{\mathbf{c}}_{ij}\}_{j \in \mathcal{N}_i}, \{\bar{\mathbf{c}}_{ji}\}_{j \in \mathcal{N}_i}, \\ & \{\boldsymbol{\lambda}_{ij}\}_{j \in \mathcal{N}_i}, \{\boldsymbol{\mu}_{ij}\}_{j \in \mathcal{N}_i}), \end{aligned} \quad (15)$$

where $\boldsymbol{\lambda}_{ij}$ and $\boldsymbol{\mu}_{ij}$ are Lagrange multipliers, $\rho > 0$ is a penalty parameter and $\mathcal{L}_i(\cdot)$ has been defined. The standard ADMM would now proceed with a step in which $\mathcal{L}_i(\cdot)$ is minimized with respect to the primal variables $(\hat{\mathbf{x}}_{Li}, \hat{\mathbf{u}}_{Li}, \hat{\mathbf{x}}_i, \hat{\mathbf{u}}_i)$ subject to local constraints, followed by a step in which $\mathcal{L}_i(\cdot)$ is minimized with respect to the other primal variables $\bar{\mathbf{c}}_{ij}$, concluding with a coordination step which updates the dual variables $\boldsymbol{\lambda}_{ij}$ and $\boldsymbol{\mu}_{ij}$. However, it can be showed after lengthy calculations (see [28] for the details) that, after a change of coordinates of the dual variables $\mathbf{q}_i = \sum_{j \in \mathcal{N}_i} (\boldsymbol{\lambda}_{ij} + \boldsymbol{\mu}_{ij})$, the algorithm can be simplified to the following two steps: 1) each robot solves a local constrained optimization problem

$$\begin{aligned} \begin{bmatrix} \bar{\mathbf{u}}_i^{k+1} \\ \hat{\mathbf{u}}_{Li}^{k+1} \\ \hat{\mathbf{x}}_{Li}^{k+1} \\ \bar{\mathbf{x}}_i^{k+1} \end{bmatrix} &= \arg \min_{\substack{\mathbf{u}_i, \hat{\mathbf{u}}_{Li} \\ \hat{\mathbf{x}}_{Li}, \bar{\mathbf{x}}_i}} \left\{ J_i(\hat{\mathbf{x}}_{Li}, \hat{\mathbf{u}}_{Li}, \hat{\mathbf{x}}_i, \bar{\mathbf{u}}_i) + \mathbf{q}_i^{k\top} \hat{\mathbf{u}}_{Li} \right. \\ &\quad \left. + \rho \sum_{j \in \mathcal{N}_i} \left\| \hat{\mathbf{u}}_{Li} - \frac{1}{2} (\hat{\mathbf{u}}_{Li}^k + \hat{\mathbf{u}}_{Lj}^k) \right\|^2 \right\} \\ &\text{subject to local constraints from (13),} \end{aligned} \quad (16)$$

where the exponent k represents the iteration count of the ADMM algorithm; 2) the local variable $\hat{\mathbf{u}}_{Li}^{k+1}$ is communicated to neighboring robots (and the one of the neighbors is received) and the following dual update step is performed:

$$\mathbf{q}_i^{k+1} = \mathbf{q}_i^k + \rho \sum_{j \in \mathcal{N}_i} (\hat{\mathbf{u}}_{Li}^{k+1} - \hat{\mathbf{u}}_{Lj}^{k+1}), \quad (17)$$

where $\mathbf{q}_i$ can be interpreted as the integral of the consensus error. Notice that the load input trajectory $\hat{\mathbf{u}}_{Li}^{k+1}$ is the only information which needs to be communicated to neighboring robots. We remark that the consensus can be arbitrarily reached on velocities or pose variables, or even both (at the price of higher communication load). However, performing the consensus on the orientation trajectory creates an additional complexity related to the non-Euclidean nature of the manifold, which can be avoided by performing the consensus on the angular velocity instead.

C. Robots Inner-Loop

The DN MPC outputs the current optimization variables $(\bar{\mathbf{x}}_i, \bar{\mathbf{u}}_i)$ for each robot, computed via (16). From these, each robot extracts its reference trajectory—position $\mathbf{p}_i^r$, velocity $\mathbf{v}_i^r$, and acceleration $\mathbf{a}_i^r$ (see Fig. 3)—which are tracked by onboard low-level controllers handling full system dynamics.

Various low-level controllers can be used, such as the geometric controller from [33] (used here) or the incremental nonlinear dynamic inversion method in [34]. These map reference trajectories to motor commands, ensuring that the robots closely track the desired trajectory while respecting the physical constraints of the system, such as thrust limitations and actuator dynamics.

This hierarchical architecture decouples the high-level NMPC—based on a simplified kinematic model—from the full robot dynamics, improving computational efficiency. Although the NMPC cannot guarantee full dynamic feasibility, this is

mitigated by imposing velocity and acceleration constraints consistent with the robots' physical limits, ensuring the trajectories remain trackable.

IV. RESULTS

This section presents results from both realistic simulations in Gazebo and real-world experiments. We first evaluate the proposed algorithm in simulation, comparing it to a centralized kinematic NMPC and showcasing its obstacle avoidance capabilities. We then report real-world experimental results for different reference trajectories.

The controller uses the following parameters: the penalty coefficients for the consensus constraint violations are set to $\rho_p = 20$ for the position and $\rho_\omega = 10$ for the angular velocity. The weight matrices are diagonal, with the following values on the diagonal: $P_p = \text{diag}(200, 200, 200)$, $P_q = \text{diag}(200, 200, 200)$, $P_v = \text{diag}(8, 8, 8)$, $P_\omega = \text{diag}(0.1, 0.1, 0.1)$, $P_i = \text{diag}(1, 1, 1)$, $P_{\alpha_i} = \text{diag}(0.01, 0.01, 0.01)$, $P_{\omega_{\alpha_i}} = \text{diag}(0.01, 0.01, 0.01)$, and $P_{u_i} = \text{diag}(1.0, 1.0, 1.0)$. The maximum acceleration norm is set to $a_{\max} = 6 \text{ m/s}^2$, while $\underline{\alpha}_i = \pi/4$ and $\bar{\alpha}_i = 4/6\pi$.

A. Experimental and Simulation Setup

To validate and demonstrate the performance of the proposed controller, simulation and experimental tests were also performed with the Fly-Crane system using three quadrotors in an indoor environment. The simulations were designed to closely replicate the experimental setup through a high-fidelity Software-in-the-Loop (SITL) configuration in Gazebo. In particular, the simulation incorporates the same parameters and software stack used on the real robots—including the state estimator, control architecture, and even the firmware running on the Electronic Speed Controllers (ESCs). Cable dynamics are modeled using a series of piecewise mass-spring-damper elements, ensuring realistic behavior of the suspended load. This careful design minimizes the gap between simulation and reality, providing a reliable testbed for validating the proposed controller.

Each quadrotor weighs 1.3 kg, the load weighs 0.338 kg, and the length of each cable is 1.1 m. The aerial vehicles are equipped with standard flight controllers, four brushless motor controllers to regulate propeller speed in a closed-loop configuration, and an onboard PC running a state estimator and the geometric controller [33]. State estimation is performed onboard at 1 kHz using an Unscented Kalman Filter (UKF), which fuses Motion Capture (MoCap) system measurements (120 Hz) with IMU data (1 kHz). Each cable angle α_i and the corresponding angular velocity ω_{α_i} are determined based on the robot's position and velocity, along with the load's pose, velocity twist, and attachment point positions. All simulations and experiments are conducted using the open-source framework TeleKyb3, which provides real-time control and state estimation capabilities².

During the initial phase of each experiment, the load is lifted off the ground, and the system is brought to the initial desired configuration. In this phase, the aerial vehicles are independently

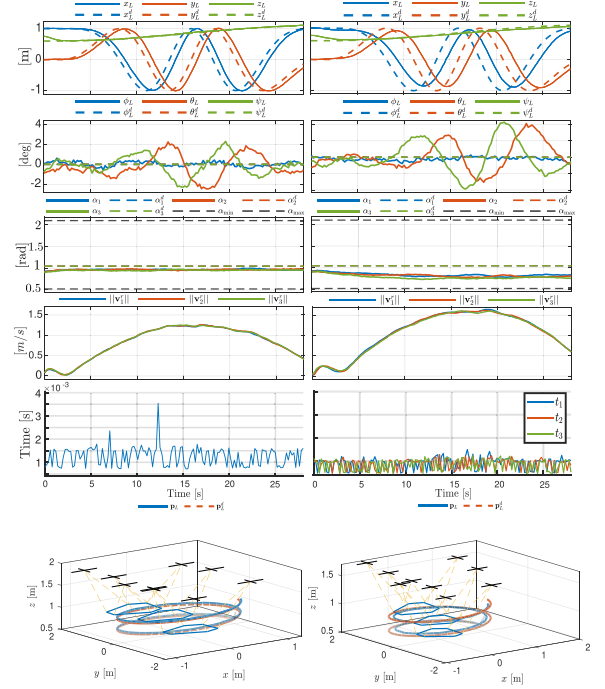


Fig. 4. Comparison performance between results obtained using centralized (on the left column) and distributed (on the right column) NMPC.

controlled using a position controller. Once stabilized, the proposed distributed controller is activated.

The proposed distributed MPC controller is implemented in MATLAB using ACADOS [35], employing a real-time iteration scheme to solve the local primal step (16) of the NMPC. Although the algorithm is designed for distributed execution, our current implementation runs one instance of the distributed controller per agent on a single desktop PC. Each instance sends the desired position, velocity, and acceleration to the corresponding quadrotor's low-level controller at 30 Hz via a Wi-Fi connection. This setup, commonly adopted in related literature, enables a fast validation of the proposed distributed scheme. In the case of the Fly-Crane, involving three robots, we consider a fully connected communication graph. Although our tests are conducted on this simpler setup, they still serve as a valid proof of concept and demonstrate the effectiveness of our approach in real experiments. Extending the implementation to a sparser communication network across multiple drones would require extensive engineering efforts, which we leave as future work.

B. Numerical Simulations

1) *Comparison With the Centralized:* We compare the proposed distributed controller with a centralized counterpart solving (8). The reference trajectory involves a spiral motion along the vertical axis, with load velocities reaching up to 1.5 m/s. As shown in Fig. 4, the centralized controller yields slightly better tracking performance, as expected, but at the cost of higher computation per iteration. Future work will explore how this trade-off evolves with larger robot teams and sparser communication graphs.

²TeleKyb3 is available at <https://git.openrobots.org/projects/telekyb3>.

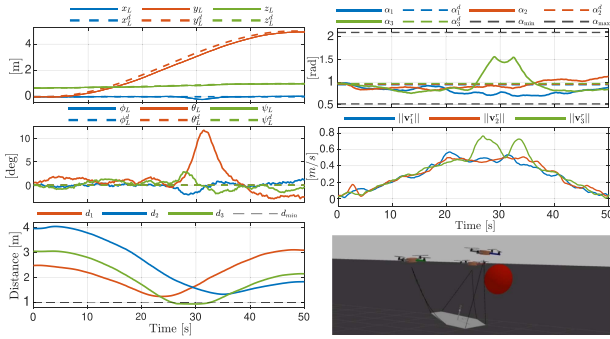


Fig. 5. Results from a simulation where obstacle avoidance intervenes to prevent robot 3 from colliding.

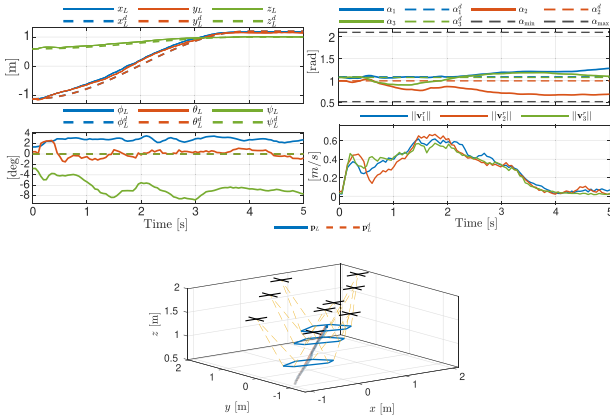


Fig. 6. Results from an experiment in which the desired trajectory is a straight line parameterized by a polynomial function.

2) *Obstacle Avoidance*: We consider a time-polynomial reference trajectory along a straight line, moving the load from approximately $[0.0, 0.0, 0.675]$ m to $[0, 5, 1]$ m, while avoiding a spherical obstacle centered at $[0.6, 3.0, 1.0]$ m with radius 0.5 m. A minimum safety distance of 1 m from the obstacle center is enforced, causing robot 3 (Fig. 5) to adjust its path accordingly.

Because the desired cable angles α_i are weighted less in the cost function, the controller prioritizes load trajectory tracking. As a result, α_3 deviates from its nominal value (see Fig. 5, upper and lower right). Still, tracking performance remains high, with only a slight pitch deviation (just over 10 deg) from the reference, while satisfying all safety constraints (lower left plot).

C. Experimental Results

We validate the proposed controller on three reference trajectories: (i) a straight line defined by a time-polynomial (Fig. 6), (ii) a trajectory that primarily induces a large rotation of the load, highlighting the controller's ability to track the full pose of the payload (Fig. 7), and (iii) a fast circular path centered at $[0, 0, 0.6]$ m with a 1.3 m radius (Figs. 8 and 9). In the straight-line and circular cases, the desired orientation is fixed at $[0, 0, 0]$ in roll, pitch, and yaw to emphasize position tracking. The aerial robots reach speeds up to 1.5 m/s during circular motion, demonstrating the controller's ability to handle dynamic trajectories. Performance is evaluated in terms of tracking accuracy for

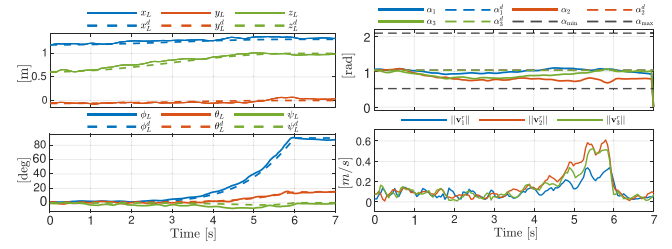


Fig. 7. Results from an experiment in which the desired trajectory requires a large yaw rotation and smaller roll and pitch rotations.

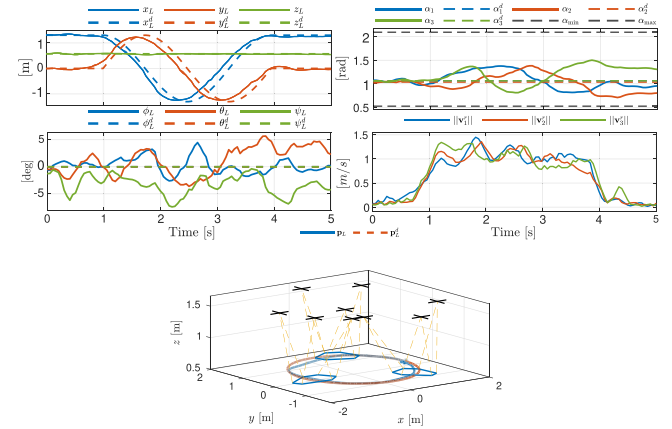


Fig. 8. Results from an experiment in which the desired trajectory is a high speed circumference.

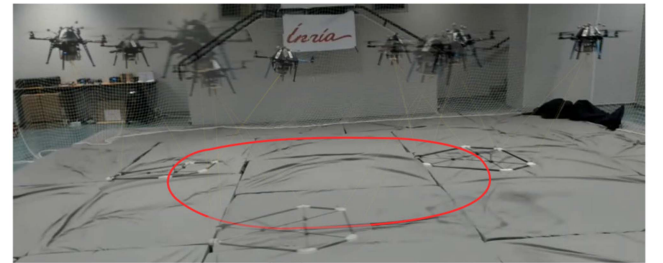


Fig. 9. View of the trajectory performed by the Fly-Crane system.

both load position and orientation. We also report cable angles α_i and robots velocity norms as estimated onboard. For the straight and circular trajectories, we include 3D tracking visualizations and snapshots of the system along the path, where transparency denotes time progression.

Position tracking errors remain within a few centimeters, mostly due to small geometric mismatches and the underactuated nature of the drones, which require tilting that affects load position. Cable-induced disturbances also contribute to minor deviations. Orientation tracking stays within 8 deg of the reference. While the tracking of the desired cable angles is assigned a low weight in the cost function—resulting in only approximate convergence to the reference setpoints—the constraints on the minimum and maximum allowable angles are always satisfied. This ensures that safe configurations are maintained throughout the motion, and potential collisions among drones are avoided.

V. CONCLUSION

In this work, we proposed a Distributed Nonlinear Model Predictive Control (DNMPC) framework for cooperative transportation with underactuated UAVs tethered to a shared payload. The method employs a partition-based ADMM algorithm to distribute computation across a peer-to-peer network, reducing communication overhead and enabling scalability. Unlike centralized formulations, each UAV solves a local subproblem, and the per-agent computational complexity remains constant regardless of team size.

The framework was validated through real-world experiments on the Fly-Crane system, demonstrating its effectiveness in achieving trajectory tracking for the full pose of the payload. Results highlighted the algorithm's robustness to system constraints and its ability to adapt to dynamic environments while maintaining high precision and satisfying physical limits.

Future work will focus on exploring the viability of extending this approach to incorporate the dynamic model of the system in the DNMPC while keeping the computation and communication complexity independent on the number of robots, exploring more efficient real-time implementations, and investigating its application in fully distributed experimental setups.

ACKNOWLEDGMENT

Experiments presented in this letter were carried out thanks to a platform of the Robotex 2.0 French research infrastructure.

REFERENCES

- [1] A. Ollero, M. Tognon, A. Suarez, D. Lee, and A. Franchi, "Past, present, and future of aerial robotic manipulators," *IEEE Trans. Robot.*, vol. 38, no. 1, pp. 626–645, Feb. 2022.
- [2] S.-J. Chung, A. A. Paranjape, P. Dames, S. Shen, and V. Kumar, "A survey on aerial swarm robotics," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 837–855, Aug. 2018.
- [3] M. Bernard, K. Kondak, I. Maza, and A. Ollero, "Autonomous transportation and deployment with aerial robots for search and rescue missions," *J. Field Robot.*, vol. 28, no. 6, pp. 914–931, 2011.
- [4] F. Augugliaro et al., "The flight assembled architecture installation: Cooperative construction with flying machines," *IEEE Control Syst. Mag.*, vol. 34, no. 4, pp. 46–64, Aug. 2014.
- [5] K. Sreenath and V. R. Kumar, "Dynamics, control and planning for cooperative manipulation of payloads suspended by cables from multiple quadrotor robots," in *Proc. Robot.: Sci. Syst.*, 2013.
- [6] A. Tagliabue, M. Kamel, R. Siegwart, and J. Nieto, "Robust collaborative object transportation using multiple MAVs," *Int. J. Robot. Res.*, vol. 38, no. 9, pp. 1020–1044, 2019.
- [7] G. Li and G. Loianno, "Nonlinear model predictive control for cooperative transportation and manipulation of cable suspended payloads with multiple quadrotors," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2023, pp. 5034–5041.
- [8] Y. Wang, G. Yu, W. Xie, W. Zhang, and C. Silvestre, "Robust cooperative transportation of a cable-suspended payload by multiple quadrotors featuring cable-reconfiguration capabilities," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 9, pp. 11833–11843, Sep. 2024.
- [9] K. Lynch, *Modern Robotics*. New York, NY, USA: Cambridge Univ. Press, 2017.
- [10] M. Manubens, D. Devaurs, L. Ros, and J. Cortés, "Motion planning for 6-D manipulation with aerial towed-cable systems," in *Proc. Robot.: Sci. Syst.*, 2013, pp. 1–26.
- [11] A. Testa, G. Carnevale, and G. Notarstefano, "A tutorial on distributed optimization for cooperative robotics: From setups and algorithms to toolboxes and research directions," *Proc. IEEE*, vol. 113, no. 1, pp. 40–65, Jan. 2025.
- [12] O. Shorinwa, T. Halsted, J. Yu, and M. Schwager, "Distributed optimization methods for multi-robot systems: Part I—a tutorial [tutorial]," vol. 31, no. 3, pp. 121–138, 2024.
- [13] H.-N. Nguyen, S. Park, J. Park, and D. Lee, "A novel robotic platform for aerial manipulation using quadrotors as rotating thrust generators," *IEEE Trans. Robot.*, vol. 34, no. 2, pp. 353–369, Apr. 2018.
- [14] T. Lee, "Geometric control of quadrotor UAVs transporting a cable-suspended rigid body," *IEEE Trans. Control Syst. Technol.*, vol. 26, no. 1, pp. 255–264, Jan. 2018.
- [15] S. Sun, X. Wang, D. Sanalidro, A. Franchi, M. Tognon, and J. Alonso-Mora, "Agile and cooperative aerial manipulation of a cable-suspended load," 2025, *arXiv:2501.18802*.
- [16] K. Klausen, C. Meissen, T. I. Fossen, M. Arcak, and T. A. Johansen, "Cooperative control for multirotors transporting an unknown suspended load under environmental disturbances," *IEEE Trans. Control Syst. Technol.*, vol. 28, no. 2, pp. 653–660, Mar. 2020.
- [17] H. G. De Marina and E. Smeur, "Flexible collaborative transportation by a team of rotorcraft," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2019, pp. 1074–1080.
- [18] A. Tagliabue, M. Kamel, S. Verling, R. Siegwart, and J. Nieto, "Collaborative transportation using MAVs via passive force control," in *Proc. 2017 IEEE Int. Conf. Robot. Automat.*, 2017, pp. 5766–5773.
- [19] C. Gabellieri, M. Tognon, D. Sanalidro, L. Pallottino, and A. Franchi, "A study on force-based collaboration in swarms," *Swarm Intell.*, vol. 14, no. 1, pp. 57–82, 2020.
- [20] D. Sanalidro, H. J. Savino, M. Tognon, J. Cortés, and A. Franchi, "Full-pose manipulation control of a cable-suspended load with multiple UAVs under uncertainties," *IEEE Robot. Automat. Lett.*, vol. 5, no. 2, pp. 2185–2191, Apr. 2020.
- [21] D. Sanalidro, M. Tognon, A. J. Cano, J. Cortés, and A. Franchi, "Indirect force control of a cable-suspended aerial multi-robot manipulator," *IEEE Robot. Automat. Lett.*, vol. 7, no. 3, pp. 6726–6733, Jul. 2022.
- [22] J. Wehbeh, S. Rahman, and I. Sharf, "Distributed model predictive control for UAVs collaborative payload transport," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2020, pp. 11666–11 672.
- [23] R. C. Sundin, P. Roque, and D. V. Dimarogonas, "Decentralized model predictive control for equilibrium-based collaborative UAV bar transportation," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2022, pp. 4915–4921.
- [24] L. Chen, H. Hopman, and R. R. Negenborn, "Distributed model predictive control for cooperative floating object transport with multi-vessel systems," *Ocean Eng.*, vol. 191, 2019, Art. no. 106515.
- [25] O. Shorinwa and M. Schwager, "Distributed contact-implicit trajectory optimization for collaborative manipulation," in *Proc. IEEE Int. Symp. Multi-Robot Multi-Agent Syst.*, 2021, pp. 56–65.
- [26] T. Erseghe, "A distributed and scalable processing method based upon ADMM," *IEEE Signal Process. Lett.*, vol. 19, no. 9, pp. 563–566, Sep. 2012.
- [27] N. Bastianello, R. Carli, L. Schenato, and M. Todescato, "A partition-based implementation of the relaxed ADMM for distributed convex optimization over lossy networks," in *Proc. IEEE Conf. Decis. Control*, 2018, pp. 3379–3384.
- [28] O. Shorinwa and M. Schwager, "Distributed model predictive control via separable optimization in multiagent networks," *IEEE Trans. Autom. Control*, vol. 69, no. 1, pp. 230–245, Jan. 2024.
- [29] M. Hamandi, M. Tognon, and A. Franchi, "Direct acceleration feedback control of quadrotor aerial vehicles," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2020, pp. 5335–5341.
- [30] L. Balandi, P. R. Giordano, and M. Tognon, "QP-based inner-loop control for constraint-safe and robust trajectory tracking for aerial robots," *IEEE Robot. Automat. Lett.*, vol. 10, no. 7, pp. 6640–6647, Jul. 2025.
- [31] D. Six, S. Briot, A. Chriette, and P. Martinet, "The kinematics, dynamics and control of a flying parallel robot with three quadrotors," *IEEE Robot. Automat. Lett.*, vol. 3, no. 1, pp. 559–566, Jan. 2018.
- [32] S. Boyd et al., "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Found. Trends Mach. Learn.*, vol. 3, no. 1, pp. 1–122, 2011.
- [33] T. Lee, M. Leok, and N. H. McClamroch, "Geometric tracking control of a quadrotor UAV on SE (3)," in *Proc. 49th IEEE Conf. Decis. Control*, 2010, pp. 5420–5425.
- [34] E. Tal and S. Karaman, "Accurate tracking of aggressive quadrotor trajectories using incremental nonlinear dynamic inversion and differential flatness," *IEEE Trans. Control Syst. Technol.*, vol. 29, no. 3, pp. 1203–1218, May 2021.
- [35] R. Verschueren et al., "acados—a modular open-source framework for fast embedded optimal control," *Math. Program. Comput.*, vol. 14, no. 1, pp. 147–183, 2022.